

requires_grad, .grad, detach & no_grad, Worked Out by Hand

Where gradients live, why they accumulate, and how to stop the graph

A complete numerical walk-through of the four switches that control PyTorch's autograd engine. We take one tiny tensor, $x = [1, 2, 3]$, and trace exactly what happens to its `.grad` buffer across repeated `backward()` calls — why the numbers *sum* instead of overwrite, why that is the reason `optimizer.zero_grad()` exists, and how `detach()` and `torch.no_grad()` cut the computation graph. Nothing is hand-waved — every number below was produced by the PyTorch reference program in Appendix A.

What this document does. It fixes a leaf tensor $x = [1, 2, 3]$ with `requires_grad=True`, defines the scalar $y = \sum_i x_i^2$, and computes $\partial y / \partial x$ by hand. It then calls `backward()` a second and third time *without zeroing*, and watches `x.grad` grow from $[2, 4, 6]$ to $[4, 8, 12]$ to $[6, 12, 18]$ — the classic accumulation bug, derived exactly.

How to read this. Each idea is isolated in its own worked step with a coloured box pinning down the one thing that matters. The accumulation arithmetic in Steps 2–3 is the heart of the document; `detach` and `no_grad` (Steps 4–5) are the two ways to keep autograd from tracking work it should not track.

Concepts covered, in order: `requires_grad` as opt-in tracking · the leaf / non-leaf distinction and `grad_fn` · one `backward()` and the gradient $2x$ · *accumulation*: why a second `backward()` adds rather than replaces · `zero_grad()` as the fix · `requires_grad` propagation (one tainted input taints the output) · `detach()`: same data, no graph · `torch.no_grad()`: no graph construction at all · a comparison table of what tracks the graph and what shares data.

1 The setup: one leaf tensor and one scalar loss

Autograd only ever differentiates a *scalar* with respect to the leaf tensors that fed it. We pick the smallest example that still shows everything: a length-3 leaf tensor and a quadratic that collapses it to a single number.

Quantity	Symbol	Value here
Leaf tensor	x	$[1, 2, 3]$
Tracking flag	<code>requires_grad</code>	<code>True</code>
Scalar output	y	$\sum_i x_i^2 = 1 + 4 + 9 = 14$
Gradient we want	$\partial y / \partial x$	$2x = [2, 4, 6]$
Where it lands	<code>x.grad</code>	a buffer attached to x

A **leaf** is a tensor you created directly (here, by `torch.tensor`), not one computed from others. Only leaves with `requires_grad=True` get a `.grad` buffer filled in by `backward()`. Everything downstream of x (y and the squares) is a *non-leaf*: it carries a `grad_fn` but no persistent `.grad`.

Intuition

Think of `requires_grad=True` as putting a tracking collar on x . From that moment, every operation touching x is recorded into a backward graph. When you call `backward()` on the final scalar, PyTorch walks that graph in reverse, multiplies the local derivatives (the chain rule), and *deposits* the result into `x.grad`. The word "deposits" is doing real work here — it means *add to*, not *overwrite*, which is the whole story of Steps 2–3.

2 Step 1 — `requires_grad`: opt-in tracking, and where the gradient lands

Not every tensor needs a gradient. Trainable parameters (weights) set `requires_grad=True`; raw data leaves it `False` (the default). For our x we opt in. Before any `backward()` call, the buffer is empty:

$$x = [1, 2, 3], \quad x.requires_grad = \text{True}, \quad x.grad = \text{None}, \quad x.is_leaf = \text{True}.$$

Now define the scalar and differentiate by hand. With $y = \sum_i x_i^2$, each partial is $\partial y / \partial x_i = 2x_i$, independent of the other entries:

$$y = x_0^2 + x_1^2 + x_2^2 = 1 + 4 + 9 = 14, \quad \frac{\partial y}{\partial x} = 2x = [2, 4, 6].$$

Calling `y.backward()` fills the buffer with exactly this vector:

$$x.grad = \frac{\partial y}{\partial x} = [2, 4, 6]$$

The number $2x_i$ is the sensitivity of the loss to a nudge in x_i : increase $x_1 = 2$ by a tiny ε and y rises by about 4ε .

Watch out

`x.grad` is `None`, not zero, until the *first* `backward()`. Code that reads `x.grad` before any backward pass (e.g. to log or clip it) will hit `None` and crash. Guard with `if x.grad is not None`, exactly as the course’s training-loop snippet does.

3 Step 2 — The accumulation bug, derived: `backward()` adds

Here is the single most common PyTorch bug. Leave x , y , and the gradient exactly as in Step 1 — `x.grad` now holds $[2, 4, 6]$. Recompute the *same* loss $y = \sum_i x_i^2$ and call `backward()` a *second* time, **without** clearing the buffer. The new backward pass computes the same $2x = [2, 4, 6]$, but instead of replacing the buffer it *adds* into it:

$$\mathbf{x.grad}_{\text{new}} = \underbrace{[2, 4, 6]}_{\text{already there}} + \underbrace{[2, 4, 6]}_{\text{this pass}} = [4, 8, 12].$$

Do it a *third* time and the same $[2, 4, 6]$ is added again:

$$[4, 8, 12] + [2, 4, 6] = [6, 12, 18].$$

After n identical backward passes with no zeroing, the buffer holds $n \cdot 2x$:

$$\mathbf{x.grad} \text{ after } n \text{ passes} = \sum_{k=1}^n 2x = n(2x)$$

<code>backward()</code> call	this pass contributes	<code>x.grad</code> after	equals
1	$[2, 4, 6]$	$[2, 4, 6]$	$1 \cdot 2x$
2	$[2, 4, 6]$	$[4, 8, 12]$	$2 \cdot 2x$
3	$[2, 4, 6]$	$[6, 12, 18]$	$3 \cdot 2x$

The same effect with a scalar makes the pattern unmistakable. Take $x = 2$ and $\text{loss} = x^2$, so $\partial \text{loss} / \partial x = 2x = 4$. Looping three times with no zeroing gives `x.grad` = 4, 8, 12 — exactly the course’s example.

Intuition

Accumulation is a *feature*, not an accident. It lets you split one logical batch across several `backward()` calls (gradient accumulation for large models) and sum the contributions for free. The price is that you must *explicitly* reset the buffer between independent optimization steps — autograd will never do it for you.

4 Step 3 — `zero_grad()`: the one line that fixes it

The cure is to zero the buffer *before* each backward pass, so every step sees a fresh $0 + 2x = 2x$ instead of a growing sum. In raw form that is `x.grad.zero_()`; in a real loop it is `optimizer.zero_grad()`,

which zeros every parameter’s buffer at once. With the reset in place, three steps of $\text{loss} = \sum_i x_i^2$ give the *correct*, constant gradient every time:

step 0 : [2, 4, 6], step 1 : [2, 4, 6], step 2 : [2, 4, 6].

Contrast the two loops side by side — identical math, one missing line:

step	without zero_grad	with zero_grad	ratio
0	[2, 4, 6]	[2, 4, 6]	1×
1	[4, 8, 12]	[2, 4, 6]	2×
2	[6, 12, 18]	[2, 4, 6]	3×

Watch out

Forgetting `zero_grad()` does not throw an error — it *silently* inflates every gradient. By step k your effective gradient is $k\times$ too large, so the optimizer takes ever-bigger steps and the loss diverges. The symptom (training explodes after a few iterations) looks like a bad learning rate; the cause is a missing one-liner. Always zero before `loss.backward()`.

5 Step 4 — Propagation: one tracked input taints the output

`requires_grad` is contagious. An operation’s result requires grad if *any* input requires grad; only then does it get a `grad_fn` (the node that knows how to send gradient backward). Take a tracked leaf a and an untracked leaf b :

$a = [1, 2, 3]$ (`requires_grad=True`), $b = [10, 20, 30]$ (`requires_grad=False`).

Two operations, two outcomes:

$c = a + b$: `c.requires_grad = True`, `c.grad_fn = AddBackward0` (tainted by a),

$d = 2b$: `d.requires_grad = False`, `d.grad_fn = None` (no tracked input).

So c joins the graph (a non-leaf with a backward function), while d stays a plain tensor. The presence of a `grad_fn` is the litmus test: it is set exactly when a tensor is part of the differentiable graph.

Mini-example — this operation in isolation

The leaf itself has *no* `grad_fn`: a is created directly, so `a.grad_fn` is `None` even though `a.requires_grad` is `True`. "Leaf with `requires_grad=True`" and "non-leaf with a `grad_fn`" are the two kinds of node in the graph — leaves are where gradient *lands* (`.grad`), non-leaves are where it *flows through* (`grad_fn`).

6 Step 5 — `detach()`: same data, graph cut

`detach()` returns a new tensor that *shares the same storage* but is sliced out of the graph: `requires_grad=False` and `grad_fn=None`. Take $x = 3$ (tracked) and $y = x^2$:

$y = x^2 = 9$, `y.grad_fn = PowBackward0`;

$y_{\text{det}} = y.\text{detach}()$: $y_{\text{det}} = 9$, `requires_grad=False`, `grad_fn=None`.

The values are identical ($9 = 9$) and they share memory, but no gradient can flow through y_{det} . The canonical use is logging: to record a loss as a plain Python number, write `loss.detach().item()`. For $\text{loss} = x^3$ at $x = 3$ that is 27.0 — safe to print, and it does not pin the graph in memory.

Watch out

`detach()` *shares data*. If you take `xd = x.detach()` and then write `xd[0] = 100`, the original x becomes `[100, 2, 3]` too — and if x is still in a live graph, you have silently corrupted the values autograd will use, often with *no* error. Detach to read, never to mutate a tensor that is still being differentiated; use `.clone()` first if you need an editable copy.

7 Step 6 — `torch.no_grad()`: build no graph at all

`detach` cuts *one* tensor out after the fact. `torch.no_grad()` is a context manager that stops the graph from being built in the *first* place: inside it, no operation records a `grad_fn`, so nothing tracks and memory is not spent holding intermediate tensors for a backward pass that will never happen. Compare the same computation $y = \sum_i x_i^2$ on a tracked $x = [1, 2, 3]$, inside versus outside the context:

```
outside: y = 14, y.requires_grad=True, y.grad_fn=SumBackward0;
inside no_grad(): y = 14, y.requires_grad=False, y.grad_fn=None.
```

The *value* is the same 14 both ways — `no_grad` changes nothing about the forward arithmetic. It only drops the bookkeeping, which is exactly what you want for inference, evaluation, and metric calculation, where you never call `backward()`.

When this is the right choice

Wrap *all* evaluation and inference code in with `torch.no_grad():`. You skip graph construction, so the forward pass is faster and uses less memory — often the difference between a validation batch fitting in memory or not. During training, gradients on; during eval/inference, always `no_grad`; for a single logged value, `.detach().item()`.

8 Tracks the graph? Shares data? — at a glance

The four switches differ along two axes: does the result stay in the differentiable graph, and does it share storage with its source?

Construct	tracks graph?	shares data?	what it is for
<code>requires_grad=True</code>	yes (opts in)	—	mark a leaf as trainable
plain op on a tracked tensor	yes (<code>grad_fn</code> set)	view/new	normal forward pass
op with no tracked input	no	view/new	data-only computation
<code>.detach()</code>	no	yes (same storage)	log / read a value safely
inside <code>torch.no_grad()</code>	no	view/new	inference, evaluation

Read the table as a decision aid: need the value but not the gradient, from a single tensor you already have? `detach`. Running a whole block (a forward pass for evaluation) with no gradients? `no_grad`. Want a parameter to *train*? `requires_grad=True`, then remember `zero_grad()` every step.

9 Equation cheat-sheet

Loss used	$y = \sum_i x_i^2$
Gradient	$\partial y / \partial x = 2x$
One backward ($x = [1, 2, 3]$)	<code>x.grad</code> = [2, 4, 6]
n backwards, no zeroing	<code>x.grad</code> = $n(2x)$
Two passes	[4, 8, 12]; three passes [6, 12, 18]
Fix	<code>x.grad.zero_()</code> / <code>optimizer.zero_grad()</code> before each pass
Propagation rule	out requires grad $\iff$ <i>any</i> input does
<code>grad_fn</code> set	iff tensor is a non-leaf in the graph
<code>detach()</code>	same data, <code>requires_grad=False</code> , <code>grad_fn=None</code>
<code>no_grad()</code>	no <code>grad_fn</code> built inside the block

Appendix A Reference implementation

Every number above is reproducible from the following program. Run it top to bottom; the printed values match the boxed results in each step.

```
import torch

# --- Step 1: requires_grad and one backward ---
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
print(x.requires_grad, x.is_leaf, x.grad)    # True True None
y = (x**2).sum()
print("y =", y.item())                        # 14.0
y.backward()
print("x.grad =", x.grad.tolist())            # [2.0, 4.0, 6.0] (= 2x)

# --- Step 2: accumulation -- backward() ADDS, it does not overwrite ---
(x**2).sum().backward()                       # second pass, no zeroing
print("x.grad =", x.grad.tolist())            # [4.0, 8.0, 12.0]
(x**2).sum().backward()                       # third pass
print("x.grad =", x.grad.tolist())            # [6.0, 12.0, 18.0] (= 3 * 2x)

# scalar version: x=2, loss=x**2, grad=2x=4 -> 4, 8, 12
xs = torch.tensor(2.0, requires_grad=True)
for step in range(3):
    (xs**2).backward()
    print(f"step {step}: xs.grad = {xs.grad.item()}")    # 4.0, 8.0, 12.0

# --- Step 3: zero_grad fixes it -> constant 2x every step ---
x.grad.zero_()
for step in range(3):
    if x.grad is not None:
        x.grad.zero_()                               # CRITICAL: clear before backward
    (x**2).sum().backward()
    print(f"step {step}: x.grad =", x.grad.tolist())    # [2,4,6] every time

# --- Step 4: requires_grad propagation / grad_fn ---
a = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
b = torch.tensor([10.0, 20.0, 30.0])              # requires_grad=False
print(a.grad_fn)                                  # None (leaf has no grad_fn)
c = a + b
print(c.requires_grad, c.grad_fn)                 # True <AddBackward0 ...>
d = b * 2.0
print(d.requires_grad, d.grad_fn)                 # False None

# --- Step 5: detach -- same data, graph cut ---
xd = torch.tensor(3.0, requires_grad=True)
yd = xd**2
yd = yd.detach()
print(yd.item(), yde.item(), yde.requires_grad, yde.grad_fn)    # 9.0 9.0 False None
print(yd.data_ptr() == yde.data_ptr())                          # True (shared storage)
print((xd**3).detach().item())                                   # 27.0 (safe to log)

# --- Step 6: no_grad -- build no graph at all ---
xn = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
yin = (xn**2).sum()
print(yin.requires_grad, yin.grad_fn)              # True <SumBackward0 ...>
with torch.no_grad():
    yout = (xn**2).sum()
```

```
print(yout.item(), yout.requires_grad, yout.grad_fn) # 14.0 False None
```

— end of document —